

CS 243, HW 1 Solution

Tianyu Cao

October 10, 2010

1.4

a

The state diagram is in the Figure 1.

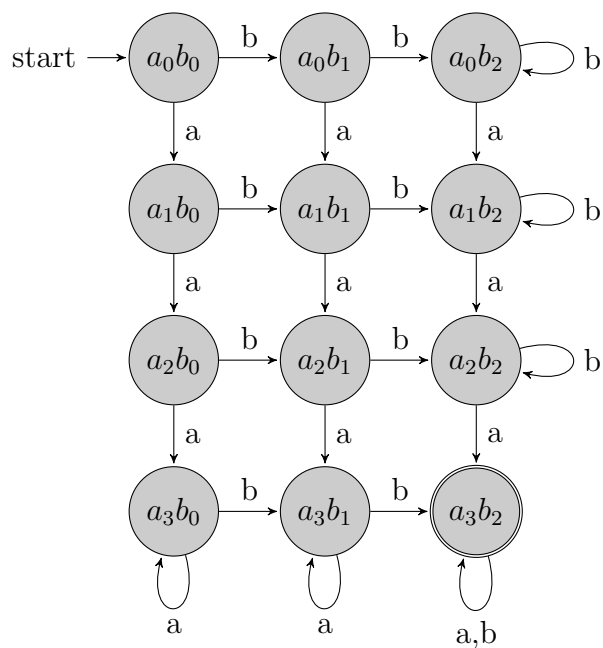


Figure 1: $w|w$ has at least three a 's and at least two b 's

c

The state diagram is in the Figure 2..

f

The state diagram is in the Figure 3.

1.6

g

The state diagram is in the Figure 4.

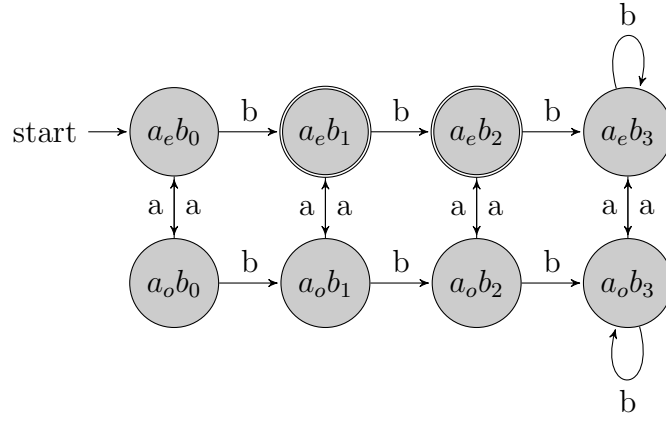


Figure 2: $w|w$ has even a 's and one or two b 's

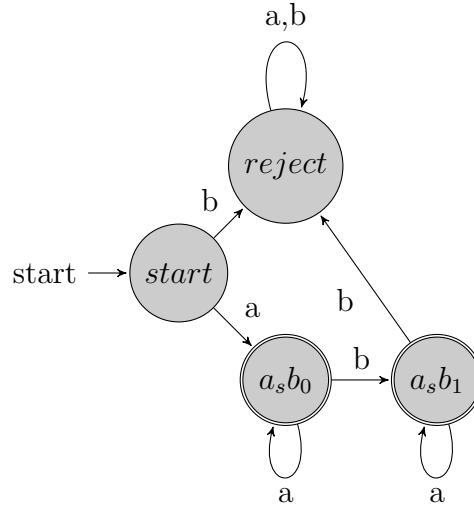


Figure 3: $w|w$ starts with an a 's and has at most one b

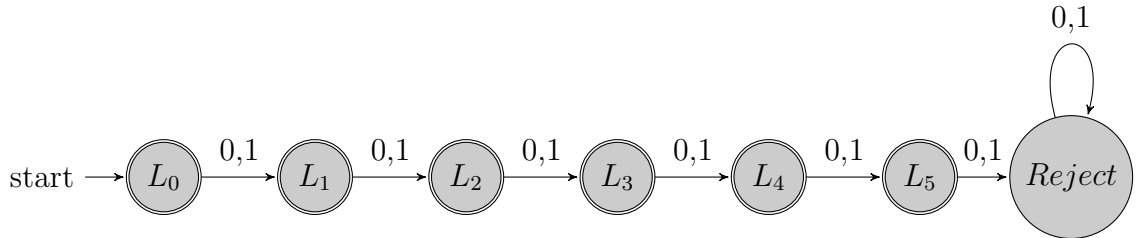


Figure 4: $w|w$ the length of w is at most 5

i

The state diagram is in the Figure 5.

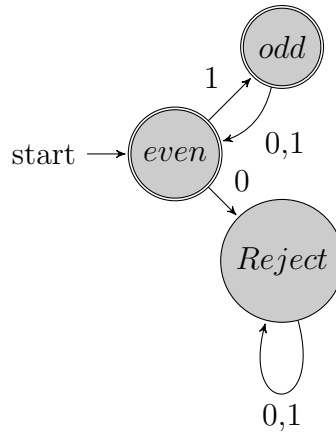


Figure 5: $w|w$ every odd position of w is a 1

k

The state diagram is in the Figure 6.

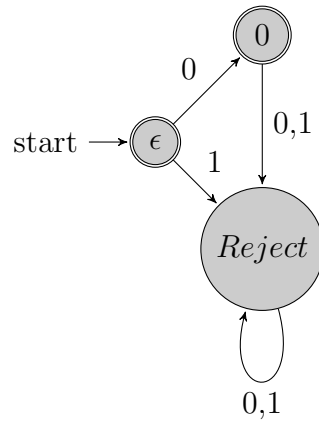


Figure 6: $w|w$ starts with an a 's and has at most one b

l

The state diagram is in the Figure 7.

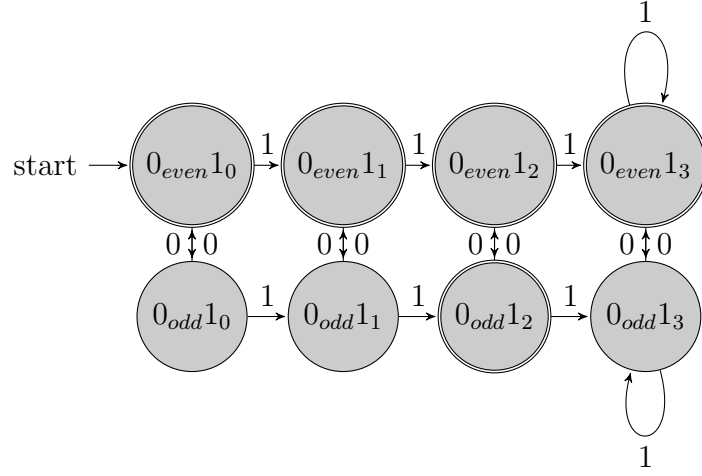


Figure 7: $w|w$ has even 0's or exactly two 1's

1.14

a

Let $M = (Q, \Sigma, \delta, q_0, F)$ is a DFA that recognizes the language B . Now, by swapping the accept and non-accept states in M , we get:

$$M' = (Q, \Sigma, \delta, q_0, Q - F).$$

We'll show that M' recognizes the complement of B .

When $w \in B$:

If $w = w_1 w_2 w_3 \cdots w_n \in B$, then by definition, there exists sequence of states $r_0, r_1, \cdots, r_n$ in Q such that:

- $r_0 = q_0$.
- $\delta(r_i, w_{i+1}) = r_{i+1}$.
- $r_n \in F$.

But, from the definition of M' , r_n is then a non-accept state which means that M' will reject the string.

When $w \notin B$:

Similarly, if $w = w_1w_2w_3 \cdots w_n \notin B$, then, the sequence of corresponding states $r_0, r_1, \dots, r_n$ in Q will result in non-accepted states for M . In other words,

- $r_0 = q_0$
- $\delta(r_i, w_{i+1}) = r_{i+1}$.
- $r_n \notin F$.

Then, as $r_n \in F - Q$, M' will accept the string.

So, M' will recognize the complement of language B . Also, it shows that if a language is regular, then its complement will also be recognized by a DFA. So, the class of regular language is closed under complement.

b

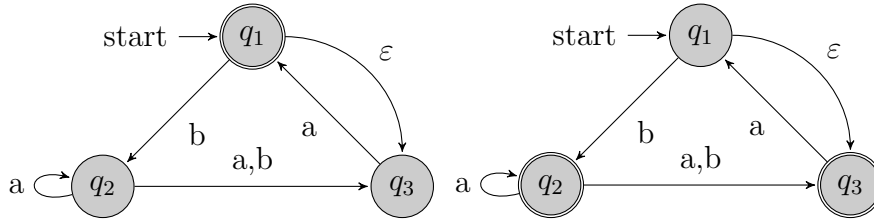


Figure 8: Swapping the accept and nonaccept states of a NFA doesn't necessarily yield a new NFA that recognizes the complement language.

The NFA in the left of Figure 8 accepts the string $\varepsilon, a, baba$ and baa but rejects b, bb and $babba$. By swapping the accept and non-accept states, we get the NFA on the right which also accepts ε but rejects $babba$ as well. So, it is apparent that resultant NFA by swapping accept and nonaccept states, would not necessarily recognizes the complement language.

But, the class of languages recognized by NFAs are still closed under complement. Because, we know that each every non-deterministic finite automation has an equivalent deterministic finite automation. And, as we have shown earlier, the class of regular languages is closed under complement — so the class of languages recognized by NFAs are also closed under complement.

1.17

a

The NFA to recognize the language $(01 \cup 001 \cup 010)^*$ is shown in the Figure 9.

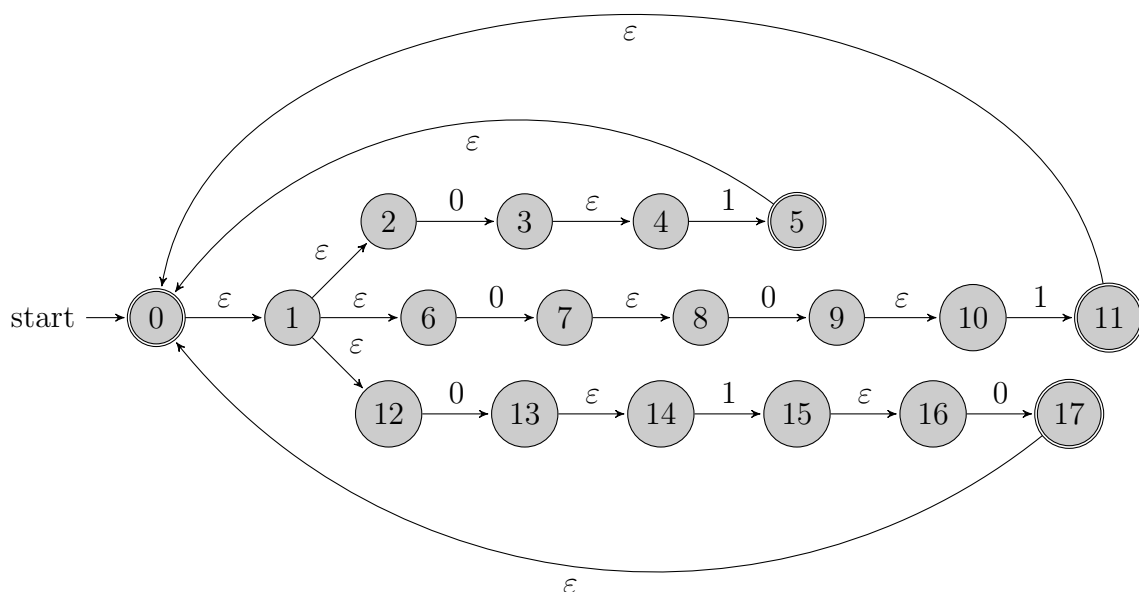


Figure 9: NFA to recognize $(01 \cup 001 \cup 010)^*$

b

The portion of DFA to recognize the language $(01 \cup 001 \cup 010)^*$ is shown in the Figure 10. Note that s_0 denotes the set of nodes $\{0, 1, 2, 6, 12\}$ in the original NFA. s_1 denotes the set of nodes $\{3, 4, 7, 8, 13, 14\}$ in the original NFA. s_2 denotes empty set $\emptyset$.

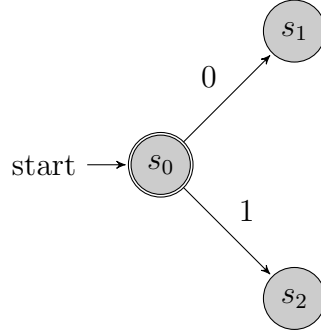


Figure 10: portion of DFA to recognize $(01 \cup 001 \cup 010)^*$

1.29

b

$$A_2 = \{www \mid w \in \{a, b\}^*\}$$

Let A_2 is regular and p is the pumping length. Let the string s be $a^p b a^p b a^p b$. As $s \in A_2$ and has length more than p , the pumping guarantees that s can be split into three pieces $s = xyz$ where for any $i \geq 0$ the string $xy^i z$ will be in A_2 .

From condition 3 of the pumping lemma, $|xy| \leq p$, so y must contain only as . So $y = a^t$ for some $t > 0$. $xyyz = a^{p+t} b a^p b a^p b$. But then, $a^{p+t} b a^p b a^p b$ will be have mismatched as , so $xyyz \notin A_2$, which contradicts our previous assumption that s can be pumped.

So A_2 can't be regular.

1.32

$B = \{w \in \Sigma_3^* \mid \text{the bottom row of } w \text{ is the sum of the two rows.}\}$

To show that B is regular, we just need to show that B^R is regular — it is accepted by a DFA. To do so, we just need to build a finite automation which will sum up the digits and compare it with the resultant lower bits. It also has to distinguish between carry and no-carry bits.

The resultant DFA which recognizes B^R is shown in Figure 11.

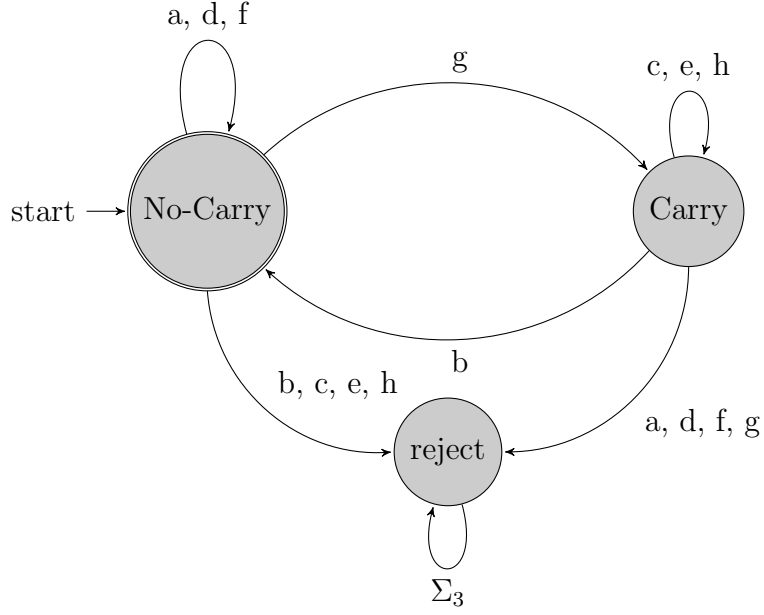


Figure 11: DFA to recognize $B = \{w \in \Sigma_3^* \mid \text{the bottom row of } w \text{ is the sum of the top two rows.}\}$

Where,

$$\begin{aligned}
 a &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, b = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \\
 c &= \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, d = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \\
 e &= \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, f = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \\
 g &= \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, h = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}
 \end{aligned}$$

1.33

$C = \{w \in \Sigma_2^* \mid \text{the bottom row of } w \text{ is three times the top row.}\}$

To show that C is regular, as before, we just need to show that C^R is regular. We'll build a DFA that recognizes C^R . To do so, we just need to build a finite automation which will multiply the top digit by 3 and compare it with the resultant lower bits. It also has to distinguish between carry and no-carry bits. However there is a difference between 1.33 and 1.32, there can be two types of carries: carry 1 and carry 2.

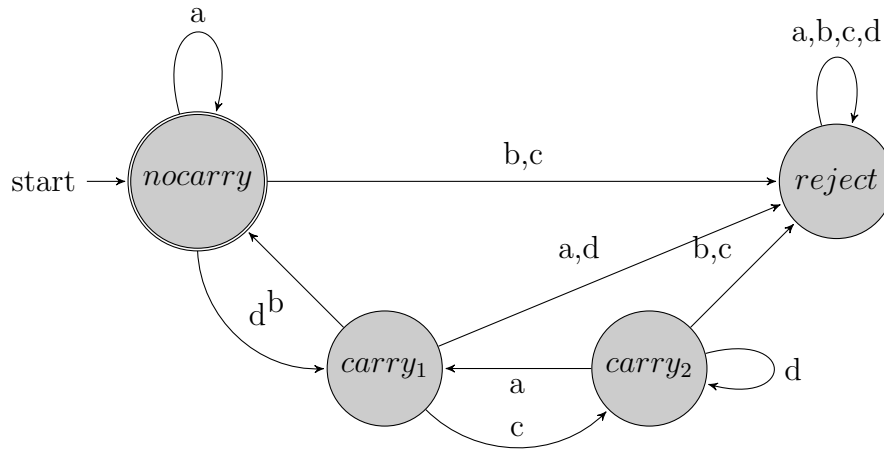


Figure 12: DFA to recognize $C = \{w \in \Sigma_2^* \mid \text{the bottom row of } w \text{ is three times the top row}\}$.

Figure 12 shows the DFA which can recognize C . Here,

$$a = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$c = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, d = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

1.36

$$B_n = \{a^k \mid \text{where } k \text{ is a multiple of } n\}.$$

To show that B_n is regular, we can build a DFA. In this DFA, the initial state will be the accept state as 0 is also a multiple of n . And, then we just need to track the number of a s we have consumed so far — by going to next state after consuming an ‘a’. Now, if we end up with $n - 1$ -th states, then after consuming next ‘a’ we will go back to the start state (which is also the only accept state) and start counting from zero again. So, our DFA will be,

$$M = (Q, \Sigma, \delta, q_0, \{q_0\}).$$

With,

$$Q = \{q_0, q_1, q_2, \dots, q_{n-1}\}$$

$$\delta(q_i, a) = q_{(i+1) \bmod n}$$

1.46

a

$$A = \{0^n 1^m 0^n \mid m, n \geq 0\}$$

Let A is regular and p is the pumping length. Let the string s be $0^p 10^p$. As $s \in A$ and has length more than p , the pumping guarantees that s can be split into three pieces $s = xyz$ where for any $i \geq 0$ the string $xy^i z$ will be in A .

From condition 3 of the pumping lemma, $|xy| \leq p$, so y must contain only 0s. So $y = 0^t$ for some $t > 0$. But then, $xyyz = 0^{p+t} 10^p$ will be have mismatched zeroes, so $xyyz \notin A$, which contradicts our previous assumption that s can be pumped.

So A can't be regular.

c

$$A = \{w \mid w \in \{0, 1\}^* \text{ is not a palindrome.}\}$$

Let A is regular. Because we know that regular language is closed under the complement operation. Let language A' be the complement of Language A . Then

$$A' = \{w \mid w \in \{0, 1\}^* \text{ is a palindrome.}\}$$

should also be a regular language. So there is a pumping length for A' . Suppose p is the pumping length. Let the string s be 0^p10^p . As $s \in A'$ and has length more than p , the pumping guarantees that s can be split into three pieces $s = xyz$ where for any $i \geq 0$ the string xy^iz will be in A' . So xy^iz will also be a palindrome.

From condition 3 of the pumping lemma, $|xy| \leq p$, so y must contain only 0s. So $y = 0^t$ for some $t > 0$. Therefore $xyyz = 0^{p+t}10^p$. The backward of $xyyz$ is 0^p10^{p+t} . Because $t > 0$, $0^p10^{p+t} \neq 0^{p+t}10^p$. This contradicts the previous conclusion that xy^iz is palindrome.

So the assumption is wrong, A can't be regular.

c

$$A = \{wtw | w, t \in \{0, 1\}^+\}$$

Let A is regular and p is the pumping length. Let the string s be 0^p110^p1 . As $s \in A$ and has length more than p , the pumping guarantees that s can be split into three pieces $s = xyz$ where for any $i \geq 0$ the string xy^iz will be in A .

From condition 3 of the pumping lemma, $|xy| \leq p$, so y must contain only 0s. So $y = 0^t$ for some $t > 0$. Therefore $xyyz = 0^{p+t}110^p1$. If $xyyz$ is in language A , then $xyyz$ can be written in the form wtw . So w must start with 0s and end with 1s. As there are only three 1s in $xyyz$. So the middle 1 must belong to t . However in such case one cannot match w to $0^{p+t}1$ and 0^p1 at the same time since $t > 0$. So $xyyz \notin A$, which contradicts our previous assumption that s can be pumped.

So A can't be regular.

1.49

a

In order to show that

$$B = \{1^k y | y \in \{0, 1\}^* \text{ and } y \text{ contains at least } k \text{ 1s, for } k \geq 1\}.$$

is regular, we need to construct an DFA to accept language B. But it seems a little difficult since the number k can be any number that is greater or equal to 1. Let us consider the following language A .

$$A = \{1y | y \in \{0, 1\}^* \text{ and } y \text{ has at least one } 1.\}$$

$\forall x \in B$, x can be written as $1^k y$ for some $k \geq 1$. So $x = 11^{k-1}y$. Because there are at least k 1s in y and $k \geq 1$. So the number of 1s in the substring $1^{k-1}y \geq 1$. Therefore $x \in A$.

$\forall x \in A$, x can be written as $1y$, where there is at least one 1 in y . So $x \in B$ because k can be 1.

Therefore we have shown that $A \subseteq B$ and $B \subseteq A$, so $A = B$. In order to show B is regular, we can instead show A is regular.

The following DFA Figure 13 accepts language A . Therefore A is regular. So B is also regular.

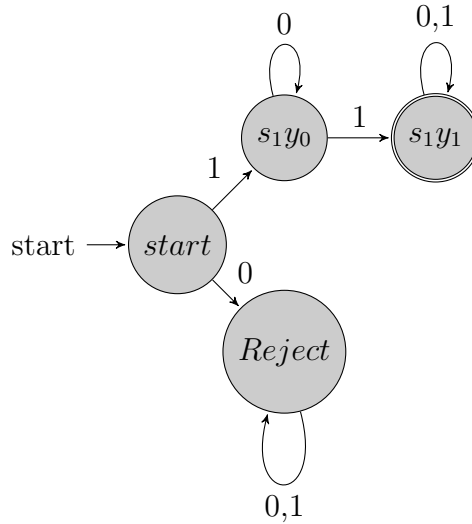


Figure 13: $w|w$ $w=1y$ and y has at least one 1.

b

$$C = \{1^k y | y \in \{0, 1\}^* \text{ and } y \text{ contains at most } k \text{ 1s, for } k \geq 1.\}$$

Let C is regular and p is the pumping length. Let the string s be $1^p 0 1^p$. As $s \in C$ and has length more than p , the pumping guarantees that s can be split into three pieces $s = uvw$ where for any $i \geq 0$ the string $uv^i w$ will

be in C .

To show that C can't be regular, we will use "pumping down" technique. The pumping lemma states that $uv^i w \in C$, even when $i = 0$. Now, we also note that $|uv| \leq p$ and $|v| > 0$. So when $i = 0$, $uw = 1^{p-t}01^p$ for some $t > 0$. If uw is in the language C , $uw = 1^k y$ for some k where the number of 1s in y is less than or equal to k . So $1^k y = 1^{p-t}01^p$. From this we can see that y must contain the substring 01^p . So $k \leq p - t$, while the number of 1s in $y \geq p$. So the number of 1's in y is always greater than k since $t > 0$. This shows that xz is not in C . This causes the desired contradiction. So C can't be regular.